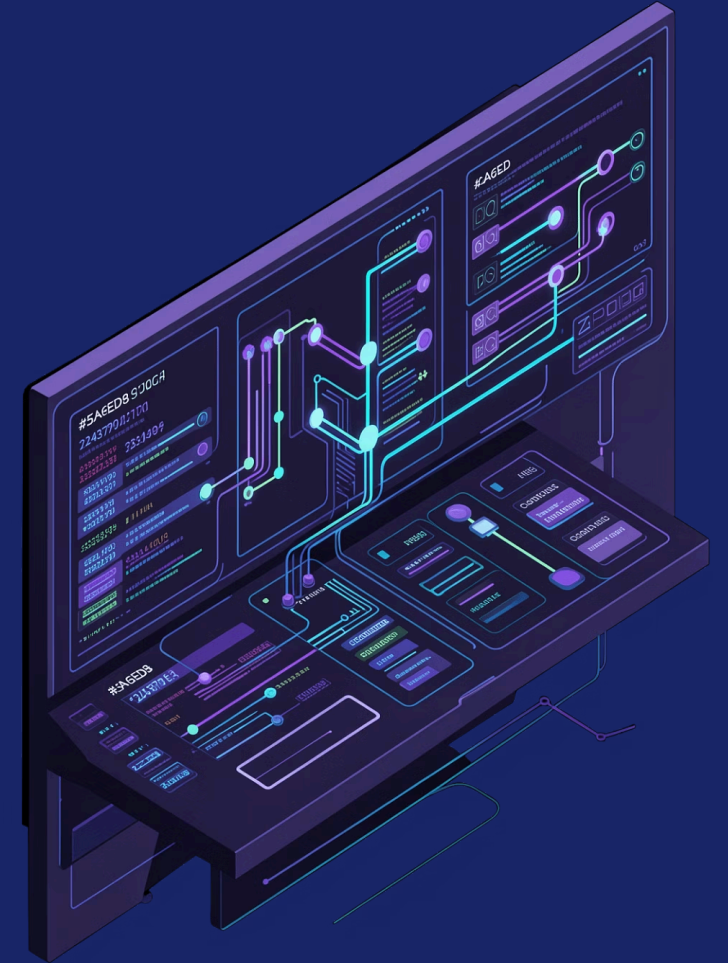


State Management

Patrón de Diseño en el Desarrollo de Software



Introducción

En el desarrollo de software moderno, una de las mayores fuentes de complejidad es la **gestión del estado** de una aplicación. Cuando una aplicación crece en tamaño y funcionalidad, coordinar los datos que cambian con el tiempo se vuelve un reto considerable.

El patrón de diseño conocido como **State Management** surge como respuesta a este problema. Su objetivo es organizar, centralizar y controlar el flujo de datos dentro de una aplicación de manera predecible y estructurada.



¿Qué es el State Management?

El Estado (State)

El **estado** de una aplicación es el conjunto de datos que describen la situación actual del sistema en un momento dado:

- El usuario está autenticado o no.
- La lista de productos cargados en pantalla.
- El número de notificaciones pendientes.
- El tema visual seleccionado (oscuro o claro).

State Management define...

Dónde

Se almacena el estado de la aplicación.

Cómo se lee

O accede a ese estado desde los componentes.

Cómo se modifica

De forma controlada y predecible.

Cómo se notifica

A los componentes cuando el estado cambia.

Conceptos Clave



Store

Contenedor central donde vive el estado global. Es la **fuentes única de verdad** (single source of truth).



Actions

Instrucciones o eventos que indican que algo cambió, como "el usuario hizo login" o "se agregó un producto al carrito".



Reducers / Mutaciones

Funciones puras que reciben el estado actual y una acción, y devuelven un **nuevo estado**. Nunca modifican el estado directamente.



Selectors

Funciones que extraen partes específicas del estado para que un componente solo acceda a los datos que necesita.

Tipos de Estado

No todo el estado en una aplicación es igual. Se puede clasificar en cuatro categorías principales:



Estado local

Datos que solo interesan a un componente específico (por ejemplo, si un menú desplegable está abierto).

Estado global

Datos que múltiples componentes necesitan acceder (por ejemplo, el usuario autenticado).

Estado del servidor

Datos provenientes de una API que se cachean localmente (React Query, SWR).

Estado de URL

Información codificada en la URL de la página (página actual, filtros aplicados).

Analogía: El Tablero de un Aeropuerto

1

El Tablero Central = Store

Contiene toda la información actualizada sobre los vuelos. Es la fuente única de verdad para todos.

2

Pasajeros y Personal = Componentes

Cada uno consulta el tablero para tomar decisiones, pero nadie lo modifica directamente.

3

Retraso de Vuelo = Action + Reducer

La aerolínea envía un evento al sistema central, que procesa la acción y actualiza el tablero.

4

Actualización Automática = Notificación

Todos los que dependen del tablero reciben la información actualizada automáticamente.



Ventajas del State Management



Fuente única de verdad

Todos los componentes comparten el mismo estado.

Predecibilidad

Los cambios de estado son trazables y controlados.

Facilidad de depuración

Herramientas como Redux DevTools permiten inspeccionar cada cambio.

Escalabilidad

La arquitectura soporta aplicaciones grandes sin perder orden.

Separación de responsabilidades

La lógica de negocio se separa de la interfaz.

Desventajas del State Management

Curva de aprendizaje

Conceptos como actions, reducers y stores pueden resultar complejos al inicio.

Código repetitivo

En soluciones como Redux se escribe mucho código (boilerplate) por cada operación.

Sobreingeniería

En aplicaciones pequeñas puede ser innecesario y agregar complejidad sin beneficio real.

Configuración inicial

Mayor tiempo de configuración inicial del proyecto antes de poder desarrollar funcionalidades.

Legibilidad

Puede dificultar la lectura del código si no se siguen buenas prácticas de organización.

¿Cuándo Usar (o No) State Management?

✓ Se recomienda cuando...

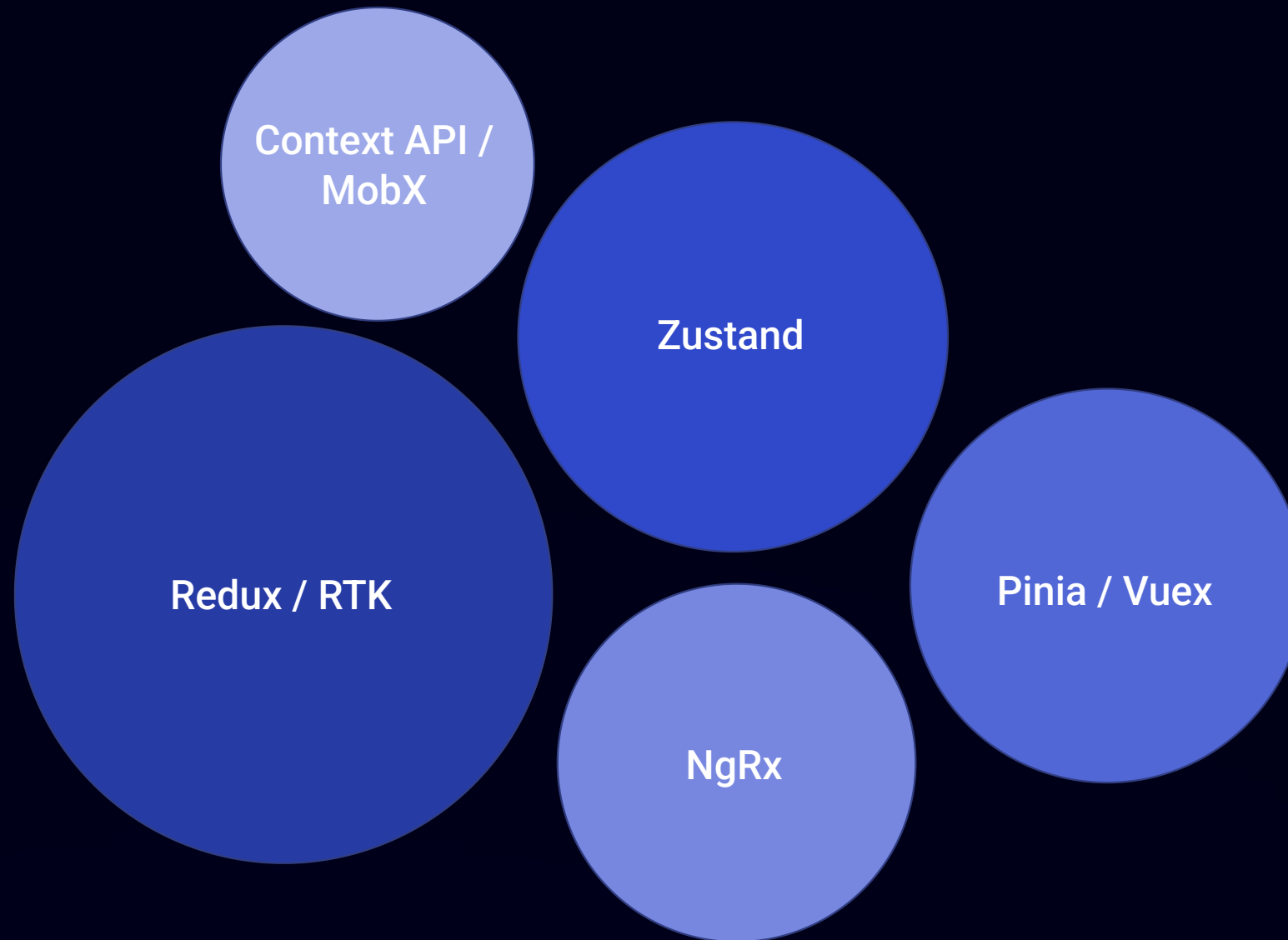
- La aplicación tiene múltiples vistas o pantallas que comparten datos.
- Muchos componentes necesitan acceder o modificar el mismo estado.
- La aplicación maneja flujos de autenticación, permisos y roles de usuario.
- Se requiere persistencia del estado entre sesiones o navegaciones.

✗ No es necesario cuando...

- Aplicaciones de una sola pantalla con lógica simple.
- Prototipos o pruebas de concepto de corta vida.
- El estado local de cada componente es suficiente para el caso de uso.

- La clave está en evaluar la complejidad real de la aplicación antes de adoptar una solución de State Management.

Herramientas y Soluciones Populares



Cada herramienta responde a un ecosistema y nivel de complejidad diferente. La elección depende del framework utilizado, el tamaño del proyecto y las preferencias del equipo.

Conclusión

El **State Management** es un patrón fundamental en el desarrollo de aplicaciones modernas. Elegir la estrategia correcta marca la diferencia entre un código mantenible y uno caótico.

Evalúa la complejidad

No toda aplicación necesita un sistema complejo de gestión de estado.

Elige la herramienta correcta

Redux, Zustand, Pinia, NgRx o Context API según el contexto del proyecto.

Aplica buenas prácticas

La separación de responsabilidades y la fuente única de verdad son principios clave.

